Parallel Programming HW2

114062640 謝秉宸

Implementation

Pthread

Load balancing

Pthread版本的load balancing是static的，假設有n條threads，那第一條會被分到0,n,2n個row的data，第二條則是1, n+1, 2n+1個row的data。有嘗試過使用以下版本來讓每條threads能盡量load balance，一開始是每次分派一個row，但時間並沒有太大改善，推測原因是爭奪mutex的時間延遲所致，接著改為每次分派50個row，但此時發現CPU utilization反而降至四百多，且時間大幅增加，可能是cache發生false sharing，因此最終將選用static的分配。

```
while(1){
        pthread_mutex_lock(&mutex);
        start=next_row;
        next_row+=10;
        pthread_mutex_unlock(&mutex);
        if(start>=height)
            break;
        end=start+10;
        if(end>height)
            end=height;

        for(i=start;i<end;i++){
            y0=i*((upper-lower)/height)+lower;
```

SIMD

使用SSE2將程式加速，其中因為SSE是128bit，所以一次只能平行運算兩筆數字，另外我是要兩個數字都計算完成才會繼續計算下兩個數字，有嘗試修改成如下版本，但時間反而變慢，因此最終使用這個版本。實作方式為先將x0與x1load 進__m128d運算

完之後每一輪都會將X與Y之平方和取出檢查，若超過四則將flag標為1，兩個flag都為1之後換下兩個數字

```
y0=i*((upper-lower)/height)+lower;
        for(j=0;j<width-1;j+=2){
            x0=j*dx+left;
            x1=(j+1)*dx+left;
            flag1=0;
            flag2=0;
            __m128d vec_y=_mm_set1_pd(0);
            __m128d vec_x=_mm_set1_pd(0);
            __m128d origin_y=_mm_set_pd(y0, y0);
            __m128d origin_x=_mm_set_pd(x0, x1);
            __m128d vec_temp=_mm_set1_pd(0);
            __m128d vec_2=_mm_set1_pd(2);
            __m128d vec_square=_mm_set1_pd(0);
            repeats1=0;
            repeats2=0;
            //length_squared=0;
            while (flag1==0||flag2==0) {
                // temp = x * x - y * y + x0;
                vec_temp=_mm_add_pd(_mm_sub_pd(_mm_mul_pd(vec_x, vec
_x), _mm_mul_pd(vec_y, vec_y)), origin_x);
                //y = 2 * x * y + y0;
                vec_y=_mm_add_pd(_mm_mul_pd(_mm_mul_pd(vec_2, vec_x),
vec_y), origin_y);
                //x = temp;
                vec_x=vec_temp;
                //length_squared = x * x + y * y;
                vec_square=_mm_add_pd(_mm_mul_pd(vec_x, vec_x), _mm_m
ul_pd(vec_y, vec_y));

                if(flag1==0){
                    _mm_storeh_pd(&temp1, vec_square);
                    ++repeats1;
                    if(temp1>=4||repeats1==iters)
                        flag1=1;
                }
```

```
            if(flag2==0){
                _mm_storel_pd(&temp2, vec_square);
                ++repeats2;
                if(temp2>=4||repeats2==iters)
                    flag2=1;
            }
        }
```

```
//SSE2其中一個運算完另一數馬上遞補的版本
y0=i*((upper-lower)/height)+lower;
        __m128d origin_y=_mm_set_pd(y0, y0);
        __m128d vec_2=_mm_set1_pd(2);

        pixel_0=0;
        x0_0 = pixel_0 * dx + left;
        repeats[0] = 0;

        pixel_1=1;
        x0_1 = pixel_1 * dx + left;
        repeats[1] = 0;
        int next_j=2;
        double temp_sq[2];

        __m128d vec_x = _mm_set1_pd(0);
        __m128d vec_y = _mm_set1_pd(0);
        __m128d origin_x = _mm_set_pd(x0_0, x0_1);
        __m128d vec_temp=_mm_set1_pd(0);
        __m128d vec_square=_mm_set1_pd(0);

        while(pixel_0<width || pixel_1<width){


            vec_temp = _mm_add_pd(_mm_sub_pd(_mm_mul_pd(vec_x, vec_
x), _mm_mul_pd(vec_y, vec_y)), origin_x);
            vec_y = _mm_add_pd(_mm_mul_pd(_mm_mul_pd(vec_2, vec_x), v
```

```
ec_y), origin_y);
            vec_x = vec_temp;
            vec_square = _mm_add_pd(_mm_mul_pd(vec_x, vec_x), _mm_mul
_pd(vec_y, vec_y));

            // _mm_store_pd(mem, vec) → mem[0]=low, mem[1]=high
            _mm_store_pd(temp_sq, vec_square);


            if (pixel_0 < width) {
                repeats[0]++;
                if (temp_sq[1]>=4 || repeats[0] == iters) {
                    image[i * width + pixel_0] = repeats[0];

                    if (next_j < width) {

                        pixel_0 = next_j;
                        x0_0 = pixel_0 * dx + left;
                        _mm_storel_pd(&tmp, vec_x);
                        vec_x = _mm_set_pd(0, tmp);
                        _mm_storel_pd(&tmp, vec_y);
                        vec_y = _mm_set_pd(0, tmp);
                        _mm_storel_pd(&tmp, origin_x);
                        origin_x = _mm_set_pd(x0_0, tmp);
                        repeats[0] = 0;
                        next_j++;
                    }
                    else
                        pixel_0=width;
                }
            }
            if (pixel_1 < width) {
                repeats[1]++;
                if (temp_sq[0]>=4 || repeats[1] == iters) {
                    image[i * width + pixel_1] = repeats[1];

                    if (next_j < width) {
                        pixel_1 = next_j;
```

```
                x0_1 = pixel_1 * dx + left;
                _mm_storeh_pd(&tmp, vec_x);
                vec_x = _mm_set_pd(tmp, 0);
                _mm_storeh_pd(&tmp, vec_y);
                vec_y = _mm_set_pd(tmp, 0);
                _mm_storeh_pd(&tmp, origin_x);
                origin_x = _mm_set_pd(tmp, x0_1);
                repeats[1] = 0;
                next_j++;
            }
            else
                pixel_1=width;
        }
    }
}
```

Hybrid

Load balancing

我用跟pthread幾乎一樣的方式將資料分配給各個process，因為考慮到若是dynamic可能要有不必要的MPI通訊，故直接選用static分配，另外在各process計算，直接用openMP的dynamic，確保process內也有load balance，實際測試下來也確實還蠻balance的。

```
#pragma omp parallel for schedule(dynamic)
```

SIMD

在此也用跟pthread版本類似的方式來做向量運算

```
double y0 = global_i * ((upper - lower) / height) + lower;

    int j;
    double temp, x0, x1;
    for (j = 0; j < width-1; j+=2) {
        x0 = j * ((right - left) / width) + left;
```

```
        x1 = (j+1) * ((right - left) / width) + left;
        int repeats1 = 0, repeats2=0;
        int flag1=0, flag2=0;
        double temp1, temp2;
        __m128d vec_y=_mm_set1_pd(0);
        __m128d vec_x=_mm_set1_pd(0);
        __m128d origin_y=_mm_set_pd(y0, y0);
        __m128d origin_x=_mm_set_pd(x0, x1);
        __m128d vec_temp=_mm_set1_pd(0);
        __m128d vec_2=_mm_set1_pd(2);
        __m128d vec_square=_mm_set1_pd(0);
        double length_squared = 0;
        while (flag1==0||flag2==0) {
            // temp = x * x - y * y + x0;
            vec_temp=_mm_add_pd(_mm_sub_pd(_mm_mul_pd(vec_x, vec_
x), _mm_mul_pd(vec_y, vec_y)), origin_x);
            //y = 2 * x * y + y0;
            vec_y=_mm_add_pd(_mm_mul_pd(_mm_mul_pd(vec_2, vec_x), ve
c_y), origin_y);
            //x = temp;
            vec_x=vec_temp;
            //length_squared = x * x + y * y;
            vec_square=_mm_add_pd(_mm_mul_pd(vec_x, vec_x), _mm_mul_
pd(vec_y, vec_y));

            if(flag1==0){
                _mm_storeh_pd(&temp1, vec_square);
                ++repeats1;
                if(temp1>=4||repeats1==iters)
                    flag1=1;
            }

            if(flag2==0){
                _mm_storel_pd(&temp2, vec_square);
                ++repeats2;
                if(temp2>=4||repeats2==iters)
                    flag2=1;
```

```
        }
    }
```

Optimization

1.Pthread

一開始程式實作完成跑起來需要約快800秒，後來先將部分宣告在回圈內的變數宣告移出迴圈，此時時間進步至七百出頭，接著我使用lab2助教提到的向量運算，實作完成後成績提升至四百多，隨後嘗試動態分配計算，但可能是修改之後程式多了些不必要的計算，成績提升並不顯著或甚至減慢，故後來仍使用一開始的靜態分配。

2.hybrid

一開始程式實作完成跑起來約700初，後來先將部分宣告在回圈內的變數宣告移出迴圈，此時時間進步至七百出頭，接著我使用lab2助教提到的向量運算，並將OpenMP改為dynamic，實作完成後時間降至至四百多秒。到目前為止，我的process分配的方式皆為一個連在一起的好幾行分配，但我在實際測試slow10這筆測資的時候，發現因為有某一區塊明顯需要跑較多時間，有就是說，在特定的worst case下，時間會暴增，因此最終改為跟Pthread一樣的分配方式。此時時間大幅壓縮至330秒。

Experiment& Analysis

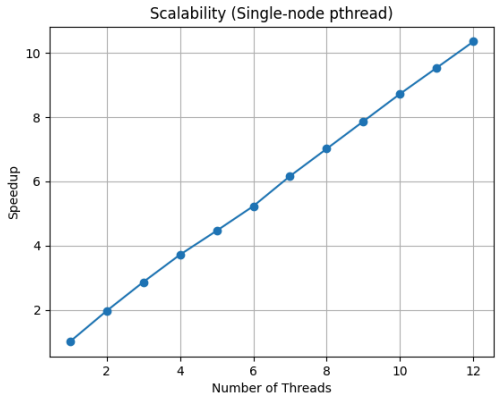
Methodology

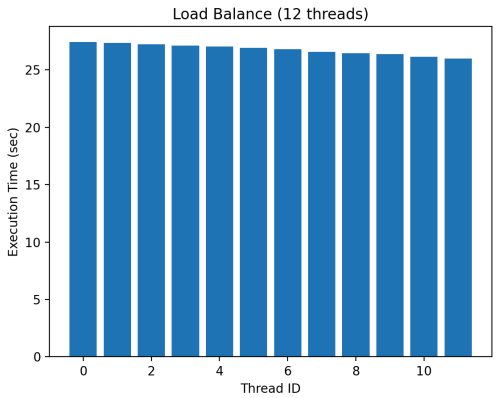
我使用課堂提供的cluster。在Pthread中用並用Clock_time紀錄時間，Hybrid則用MPI_Wtime()來記錄各階段的執行時間，

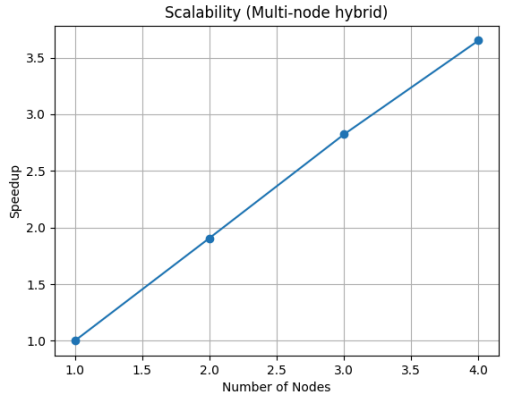
Testcase

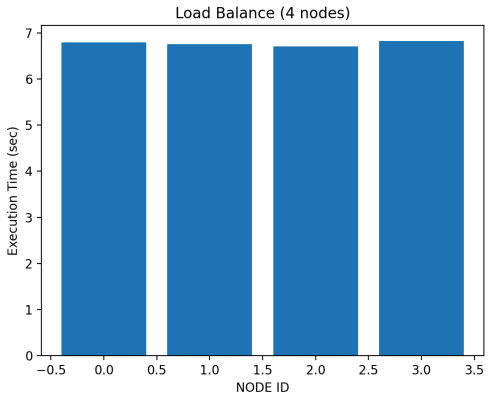
我選擇slow10作為我的測資，因為我發現這筆測資不同部分的計算量差很大，很適合拿來測試load balance。

Pthread scalability and load balance









Scalibility

由上圖可見不管是Pthread或hybrid，兩者的scability都很好，推測應該是因為大部分時間都在作計算，且不同計算之間沒有dependency。不像hw1一樣有那麼多的通訊、IO時間成本，故才能做出那麼好看的scability。

Load balance

可見兩者的load balance 都還不錯，原先我的hybrid是一整塊一整塊分給process，但在slow10這筆測資測試下會得到非常慘的load分配，之後我改為一條一條切就讓整個load都balance了。

Conclusion

做這次作業的過程相當的有成就感，因為這次是embarassing parallel，所以能做的優化非常多，我每次修改整體秒數都會進步個50~100秒，不像第一份作業有那麼多通

訊及IO成本。另外，從Hybrid的實驗中，我也深刻體認到load balance的重要性，原來要將計算量接近的部分好好分配可以那麼影響效能。